

Step 1 — The Two-Agent Pipeline, Running Standalone — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way, no config/logging yet

```
# researcher.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

RESEARCHER_SYSTEM_PROMPT = (
    "You are a careful research assistant. Given a topic, write a short, "
    "factual summary (3-5 sentences). If feedback from a previous draft is "
    "given, fix exactly the issues named in it -- do not rewrite parts that "
    "were not flagged."
)

class Draft(BaseModel):
    topic: str
    summary: str

_prompt = ChatPromptTemplate.from_messages([
    ("system", RESEARCHER_SYSTEM_PROMPT),
    ("human", "Topic: {topic}\n\nFeedback from the last fact-check (none if this "
)]
_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
_chain = _prompt | _llm | StrOutputParser()

def run_researcher(topic, feedback=None):
```

```
summary = _chain.invoke({"topic": topic, "feedback": feedback or "none"})
return Draft(topic=topic, summary=summary.strip())
```

```
# fact_checker.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

FACT_CHECKER_SYSTEM_PROMPT = (
    "You check a research summary for real factual problems: wrong dates, "
    "wrong names, invented facts, or claims stated more confidently than "
    "the evidence supports. Set approved=True only if there are no real "
    "problems. If not approved, list each specific issue to fix."
)

class FactCheckVerdict(BaseModel):
    approved: bool
    issues: list[str]

_prompt = ChatPromptTemplate.from_messages([
    ("system", FACT_CHECKER_SYSTEM_PROMPT),
    ("human", "Topic: {topic}\n\nDraft summary:\n{summary}"),
])
_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(FactCheckVerdict)
_chain = _prompt | _llm

def run_fact_checker(draft):
    return _chain.invoke({"topic": draft.topic, "summary": draft.summary})
```

This works, and it's a fine first pass to run by hand a few times. It has no type hints, no config, no logging, and creates its `ChatOpenAI` instances at import time with no way to point them at a test double — all fine to fix once you know the basic shape is right.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — full type hints, config, and the loop in `pipeline.py`

```
# config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()

@dataclass
class Config:
    log_level: str
    openai_api_key: str
    max_revision_rounds: int

def load_config() -> Config:
    return Config(
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        openai_api_key=os.getenv("OPENAI_API_KEY", ""),
        max_revision_rounds=int(os.getenv("MAX_REVISION_ROUNDS", "3")),
    )
```

```
# logging_setup.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        # StreamHandler() defaults to stderr, not stdout -- important once
        # mcp_server.py exists in Step 2, where stdout is the MCP protocol
        # channel on the stdio transport.
        handler = logging.StreamHandler()
        handler.setFormatter(logging.Formatter("%(asctime)s %(levelname)s %(name)s"))
        logger.addHandler(handler)
```

```
logger.setLevel(os.getenv("LOG_LEVEL", "INFO"))
return logger
```

```
# researcher.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

from config import load_config

config = load_config()

RESEARCHER_SYSTEM_PROMPT = (
    "You are a careful research assistant. Given a topic, write a short, "
    "factual summary (3-5 sentences). If feedback from a previous draft is "
    "given, fix exactly the issues named in it -- do not rewrite parts that "
    "were not flagged."
)

class Draft(BaseModel):
    topic: str
    summary: str

def _build_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", RESEARCHER_SYSTEM_PROMPT),
        ("human", "Topic: {topic}\n\nFeedback from the last fact-check (none if t
    ])
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2, api_key=config.openai_)
    return prompt | llm | StrOutputParser()

_chain = _build_chain()

def run_researcher(topic: str, feedback: str | None = None) -> Draft:
    summary = _chain.invoke({"topic": topic, "feedback": feedback or "none"})
    return Draft(topic=topic, summary=summary.strip())
```

```
# fact_checker.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
```

```

from langchain_core.prompts import ChatPromptTemplate

from config import load_config
from researcher import Draft

config = load_config()

FACT_CHECKER_SYSTEM_PROMPT = (
    "You check a research summary for real factual problems: wrong dates, "
    "wrong names, invented facts, or claims stated more confidently than "
    "the evidence supports. Set approved=True only if there are no real "
    "problems. If not approved, list each specific issue to fix, one per "
    "item, naming exactly what's wrong."
)

class FactCheckVerdict(BaseModel):
    approved: bool
    issues: list[str]

def _build_chain():
    prompt = ChatPromptTemplate.from_messages([
        ("system", FACT_CHECKER_SYSTEM_PROMPT),
        ("human", "Topic: {topic}\n\nDraft summary:\n{summary}"),
    ])
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key)
    return prompt | llm.with_structured_output(FactCheckVerdict)

_chain = _build_chain()

def run_fact_checker(draft: Draft) -> FactCheckVerdict:
    return _chain.invoke({"topic": draft.topic, "summary": draft.summary})

```

```

# pipeline.py
from pydantic import BaseModel

from researcher import run_researcher, Draft
from fact_checker import run_fact_checker, FactCheckVerdict

class RevisionRound(BaseModel):
    round_number: int
    summary: str

```

```

approved: bool
issues: list[str]

```

```

class PipelineResult(BaseModel):
    topic: str
    final_summary: str
    approved: bool
    rounds: list[RevisionRound]
    revision_count: int

def run_pipeline(topic: str, max_rounds: int) -> PipelineResult:
    feedback: str | None = None
    rounds: list[RevisionRound] = []
    draft: Draft | None = None
    verdict: FactCheckVerdict | None = None

    for round_number in range(1, max_rounds + 1):
        draft = run_researcher(topic, feedback=feedback)
        verdict = run_fact_checker(draft)

        rounds.append(RevisionRound(
            round_number=round_number,
            summary=draft.summary,
            approved=verdict.approved,
            issues=verdict.issues,
        ))

        if verdict.approved:
            break
        feedback = "; ".join(verdict.issues)

    return PipelineResult(
        topic=topic,
        final_summary=draft.summary,
        approved=verdict.approved,
        rounds=rounds,
        revision_count=len(rounds),
    )

```

```

# main.py
from pipeline import run_pipeline
from logging_setup import get_logger

logger = get_logger(__name__)

```

```

TEST_TOPICS = [
    "Retrieval-Augmented Generation (RAG)",
    "my opinion of the color blue", # deliberately hard to fact-check
]

def main():
    for topic in TEST_TOPICS:
        result = run_pipeline(topic, max_rounds=3)
        print(f"\n=== {topic} ===")
        for round_info in result.rounds:
            status = "approved" if round_info.approved else "rejected"
            print(f"Round {round_info.round_number}: {status}")
            for issue in round_info.issues:
                print(f"  - {issue}")
        if result.approved:
            print(f"\nFinal (approved after {result.revision_count} round(s)):")
        else:
            print(f"\nFinal (NOT approved -- hit the {result.revision_count}-round cap)")
        print(result.final_summary)

if __name__ == "__main__":
    main()

```

Expected behavior: running `python main.py` prints each round for both topics. The RAG topic should show at least one rejected round with a real, specific issue, then an approved round. The "opinion of the color blue" topic should either approve quickly (there's nothing to fact-check) or hit the 3-round cap and still print a final summary, clearly marked not approved — either is a correct result, as long as it stops at 3 rounds.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a sanity check that catches a broken loop early

```
# test_pipeline_sanity.py -- a throwaway script, run once by hand
from pipeline import run_pipeline

result = run_pipeline("Retrieval-Augmented Generation (RAG)", max_rounds=3)

# the final summary must always match the LAST round's summary --
# if it doesn't, the loop is tracking the wrong round somewhere
assert result.rounds[-1].summary == result.final_summary, (
    "final_summary doesn't match the last round -- check the loop's "
    "draft/verdict tracking"
)

# the loop must never run more rounds than the cap allows
assert result.revision_count <= 3, "loop ran past max_rounds"

# if it says approved, the last round's verdict must actually say so too
if result.approved:
    assert result.rounds[-1].approved, "approved=True but last round wasn't approved"

print("Sanity checks passed.")
print(f"Rounds: {result.revision_count}, approved: {result.approved}")
```

Expected output:

```
Sanity checks passed.
Rounds: 2, approved: True
```

(the exact round count depends on what the model does -- 1-3 is all valid, as long as it never exceeds `max_rounds`)

This is the kind of check worth running once after building the loop, and again any time you refactor it — it catches exactly the class of bug Hint 2's Advanced section warned about (returning the wrong round's draft) immediately, instead of you noticing it by accident three steps later when the MCP tool's trace looks wrong.

Approach 2 — a fake Fact-Checker, to test the loop's shape without spending real API calls

```
# test_pipeline_with_fake_checker.py -- a throwaway script
from unittest.mock import patch
from fact_checker import FactCheckVerdict
import pipeline

def fake_fact_checker(draft):
    # always rejects, no matter what -- proves the cap actually stops the loop
    return FactCheckVerdict(approved=False, issues=["always rejected, for testing"])

with patch("pipeline.run_fact_checker", side_effect=fake_fact_checker):
    result = pipeline.run_pipeline("any topic", max_rounds=3)

assert result.revision_count == 3, "loop should have run exactly max_rounds times"
assert result.approved is False, "should never be approved with a fact-checker that always rejects"
print("Cap test passed:", result.revision_count, "rounds, approved =", result.approved)
```

Expected output:

```
Cap test passed: 3 rounds, approved = False
```

This is the actual proof the "hard cap, never loops forever" requirement holds — not by reading the code and believing it, but by forcing the one case (a Fact-Checker that never approves) that would reveal a bug immediately if the `break` condition or the `range()` bound were wrong. It also costs zero real API calls, since `run_researcher` is the only thing still hitting the model, and it runs fast enough to include in a real test suite later.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate is the working pipeline itself. Approach 1 adds a quick, cheap correctness check you run against the real pipeline once, by hand. Approach 2 goes further — it isolates the loop's *shape* (does it actually stop at the cap?) from the model's actual behavior, using a fake Fact-Checker that always rejects, which is the only way to be completely certain the cap works, rather than hoping your test topics happen to hit it.

Which one should you actually write? Build the Intermediate version — it's what `researcher.py`, `fact_checker.py`, `pipeline.py`, and `main.py` should actually contain going forward. Run Advanced Approach 1 once by hand after you build it, as a real sanity check. Approach 2's fake-checker test is worth keeping if you're comfortable with `unittest.mock` — it's the only way

to test the cap with total certainty and no API cost — but skip it for now if mocking is new to you; nothing later in this project depends on it existing.